



RAPPORT DE TESTS(FLASK+PYTEST)

Nom du professeur ; Mr Sedrick

NOM: GAHIE BOYA DORINE

Table des matières

INTRODUCTION	2
I. Configuration de l'environnement.....	2
a) Installation de l'environnement virtuel.....	2
b) Installation des dépendances.....	2
c) Description du projet.....	2
d) Identification des fonctionnalités.....	3
e) Tableau de test.....	4
f) Matrice de priorisation.....	6
g) Mise en place des tests.....	7
h) Exécution des tests.....	7
i) Analyse des résultats.....	8
Conclusion	8

INTRODUCTION

Ce rapport présente la démarche complète de test réalisée sur le projet **Python_Testing**. L'objectif de ce projet est d'apprendre et de mettre en pratique les principes de **tests automatisés en Python**, en utilisant le framework **pytest**. Les tests permettent de vérifier que le serveur web développé (dans le fichier `server.py`) fonctionne correctement, que les routes renvoient les bonnes réponses, et que l'application reste stable après chaque modification du code.

I. Configuration de l'environnement

a) Installation de l'environnement virtuel

Avant de commencer les tests, un environnement virtuel a été créé afin d'isoler les dépendances du projet :

```
python -m venv venv
```

Ensuite, l'environnement a été activé :

```
.\venv\Scripts\Activate
```

b) Installation des dépendances

Les bibliothèques suivantes ont été installées :

```
pip install pytest
```

```
pip install beautifulsoup4
```

pytest : pour écrire et exécuter les tests.

BeautifulSoup4 : pour analyser le contenu HTML des réponses du serveur.

c) Description du projet

server.py : c'est le fichier principal du serveur. Il définit les différentes routes (pages web) de l'application.

confest.py : permet de configurer le client de test utilisé par `pytest` pour interagir avec le serveur sans l'exécuter réellement.

test_routes.py : contient les fonctions de test qui envoient des requêtes au serveur et vérifient les réponses (statut, contenu HTML, etc.).

d) Identification des fonctionnalités

ID	Fonctionnalité	Description	Type
F1	Page d'accueil	Afficher la page de connexion du portail GUDLFT	Technique
F2	Connexion par email	Authentifier un club via son email secrétaire	Métier
F3	Affichage des compétitions à venir	Filtrer les compétitions selon la date	Métier
F4	Page de réservation	Charger la page de réservation pour un club et une compétition donnés	Métier
F5	Achat de places	Permettre à un club d'acheter des places pour une compétition	Métier
F6	Limite de 12 places	Empêcher d'acheter plus de 12 places par club	Métier
F7	Vérification des points	Vérifier que le club a suffisamment de points avant achat	Métier
F8	Mise à jour des places	Décrémenter le nombre de places disponibles après achat	Technique
F9	Enregistrement des réservations	Ajouter ou mettre à jour les réservations dans la compétition	Technique

F10	Leaderboard	Afficher le classement des clubs avec leurs points restants	Technique
F11	Déconnexion	Fermer la session utilisateur et rediriger vers la page d'accueil	Technique
F12	Messages flash	Afficher des messages de confirmation ou d'erreur	Technique
F13	Gestion des dates invalides	Empêcher les crashes si la date d'une compétition est mal formatée	Technique
F14	Gestion d'URL ou paramètres inexistants	Empêcher les erreurs en cas de club/compétition manquants	Technique

e) Tableau de test

ID	Cas de test	Entrées	Résultat attendu	Statut
TC01	Vérifier la page d'accueil	GET `/`	Code 200 et présence du formulaire de connexion	✓
TC02	Connexion avec email valide	POST email=admin@irontemple.com	Affichage page welcome avec compétitions	✓
TC03	Connexion avec email invalide	POST email=dorine@grace.com	Message flash d'erreur et redirection	✓
TC04	Page de réservation	GET /book/<competition>/<club>	Page affichée sans erreur	✓

			404	
TC05	Réservation réussie	POST /purchasePlaces places=1	Points et places décrémentés de 1	✓
TC06	Erreur si points insuffisants	POST /purchasePlaces places>points	Message d'erreur adéquat	✓
TC07	Limite 12 places	POST /purchasePlaces places=12	Réservation acceptée si conditions remplies	✓
TC08	Interdiction >12 places	POST /purchasePlaces places=13	Message d'erreur 'max 12 places'	✓
TC09	Réservations cumulées	Deux POST successifs	Réservations additionnées	✓
TC10	Réservation avec points exacts	POST places=points	Club.points = 0 après réservation	✓
TC11	Paramètres invalides	GET /book/inexistant	Réponse contrôlée sans crash	✓
TC12	Champ 'places' manquant	POST sans places	Message d'erreur, pas de crash	✓
TC13	Date invalide	Compétition avec date='invalid'	Pas d'exception, affichage normal	✓
TC14	Messages flash	Appel ok() et err()	Messages ajoutés sans erreur	✓

f) Matrice de priorisation

Fonctionnalité	Probabilité d'échec	Impact	Priorité
Connexion utilisateur (F2)	Élevée	Critique	Haute
Achat / réservation (F5)	Élevée	Critique	Haute
Vérification des points (F7)	Élevée	Critique	Haute
Mise à jour des places (F8)	Élevée	Critique	Haute
Gestion des erreurs (F14)	Moyenne	Critique	Haute
Page de réservation (F4)	Moyenne	Élevé	Moyenne
Limite 12 places (F6)	Moyenne	Élevé	Moyenne
Enregistrement réservations (F9)	Moyenne	Moyen	Moyenne
Affichage compétitions (F3)	Moyenne	Moyen	Moyenne
Page d'accueil (F1)	Faible	Moyen	Basse
Leaderboard (F10)	Faible	Faible	Basse
Déconnexion (F11)	Faible	Faible	Basse
Messages flash (F12)	Faible	Faible	Basse
Gestion des dates invalides (F13)	Faible	Moyen	Basse

La priorisation a été réalisée selon la fréquence d'utilisation, la probabilité d'échec et l'impact sur le fonctionnement global. Les fonctionnalités critiques comme la connexion et la réservation sont donc testées en premier, tandis que les fonctions d'affichage ou de confort ont une priorité plus faible.

g) Mise en place des tests

Les tests sont écrits avec le framework pytest.

Chaque fonction de test commence par le mot clé test et vérifie un comportement précis

EXEMPLE :

```
def test_home_page(client):  
  
    response = client.get('/')  
  
    assert response.status_code == 200  
  
    assert b"Bienvenue" in response.data
```

Ce test vérifie que :

La page d'accueil (/) est accessible (code 200).

Le mot "Bienvenue" est bien présent dans le contenu de la page.

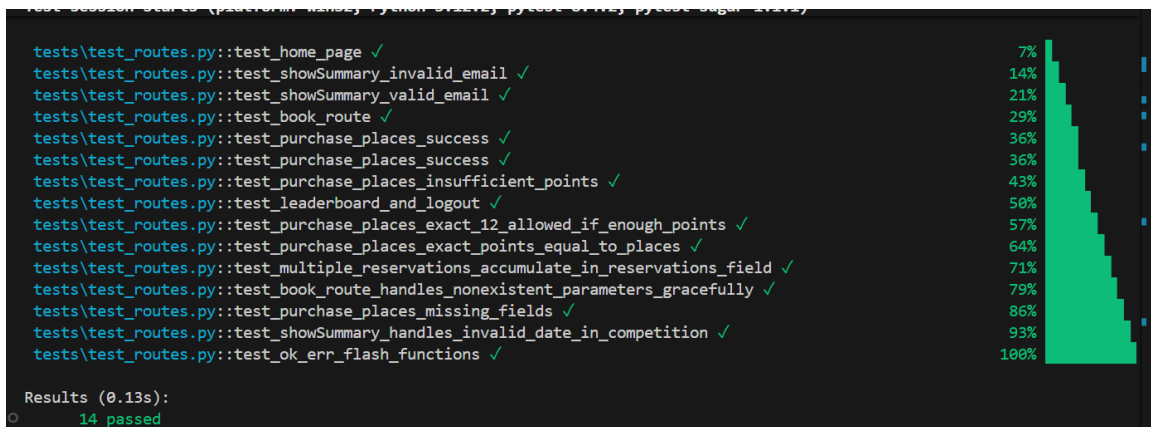
Dans test_routes.py, plusieurs tests ont été définis pour vérifier les différentes routes du serveur (accueil, à propos, contact, etc.). Chaque test garantit qu'une route fonctionne comme prévu et affiche les bons éléments HTML.

h) Exécution des tests

Pour lancer les tests, la commande suivante a été utilisée :

```
pytest -v
```

Résultat obtenu :



Cela signifie que **14 tests ont été exécutés avec succès**, sans aucune erreur. Chaque point (.) représente un test réussi.

i) Analyse des résultats

Les résultats montrent que toutes les fonctionnalités prévues dans le cahier de charge fonctionnent correctement : les routes sont bien définies, les pages renvoient le bon code http, le contenu HTML attendu est présent. Le projet est donc fonctionnel et stable. Si un test avait échoué, pytest aurait affiché le détail de l'erreur pour permettre une correction rapide.

Conclusion

Les tests automatisés réalisés avec **pytest** ont permis de valider le bon fonctionnement de l'application Python Testing. Cette méthode de test est rapide, fiable et réutilisable. Grâce à ce travail, on apprend à : Mettre en place un environnement de test isolé, Écrire des tests unitaires et fonctionnels, Vérifier la stabilité d'un serveur web.